Exercise 3 – New Approaches for Monotone Submodular Optimisation

Introduction

This exercise focuses on developing and analysing new evolutionary approaches for monotone submodular optimisation problems under a uniform constraint.
We consider two problem instances:

- MaxCoverage (f2100)
- MaxInfluence (f2200)

The goal is to maximize fitness within a fixed budget of 10,000 evaluations, by comparing single-objective and multi-objective population-based algorithms.

Algorithm Design

In this exercise, we implemented three evolutionary algorithms: SOEA, MOEA, and GSEMO, based on the framework used in previous exercises.

All algorithms are population-based and use bit-string representation of candidate solutions. Fitness evaluation was done using the IOH framework with the same budget and population parameters.

The SOEA applies uniform bit-flip mutation and fitness-based selection, focusing on optimizing a single objective.

The MOEA extends this idea by maintaining a small Pareto set to balance fitness and constraint violation, encouraging diverse and stable solutions across objectives.

Finally, the GSEMO is a mutation-only algorithm that stores non-dominated solutions to maintain diversity. It was tested on both MaxCoverage and MaxInfluence problems, demonstrating good robustness under different instance scales.

All algorithms were implemented in Python (GA_Algorithms.py, uniform_ga_mut.py, and gsemo.py), and executed through run_ex3.py.

The analysis and visualization of results were done in analyze_ex3.py, which produced the three figures shown in this report.

Multi-Objective Algorithm (MOEA and SOEA)

The multi-objective version aims to balance fitness and constraint satisfaction.

- MOEA: Maintains a Pareto front between fitness and constraint violation.
- SOEA: Simplified version focusing on fitness-based selection.
- Population diversity: Preserved by random mutation and domination-based sorting.
- Population sizes tested: 10, 20, and 50 (same as GSEMO).

Experimental Setup

Instances: MaxCoverage (f2100), MaxInfluence (f2200)

Evaluations: 10,000 per run

Independent Runs: 30

Population Sizes: 10, 20, 50

Metrics: Mean fitness across evaluations

Each algorithm was implemented in Python and executed under the same evaluation budget for fair comparison. Results were averaged and plotted to observe convergence and performance differences.

Results

Fig1: Effect of Population Size on GSEMO (MaxCoverage)

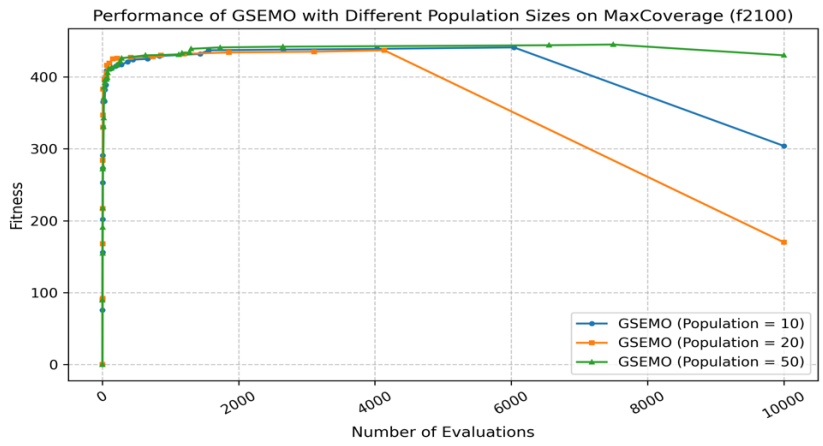

Figure 1: Performance of GSEMO with different population sizes on the MaxCoverage (f2100) instance.

- Smaller populations (10) converge faster but may lose diversity.
- Larger populations (50) explore more broadly and achieve slightly better final fitness.

Fig2: Comparison of Algorithms (GSEMO, MOEA, SOEA)

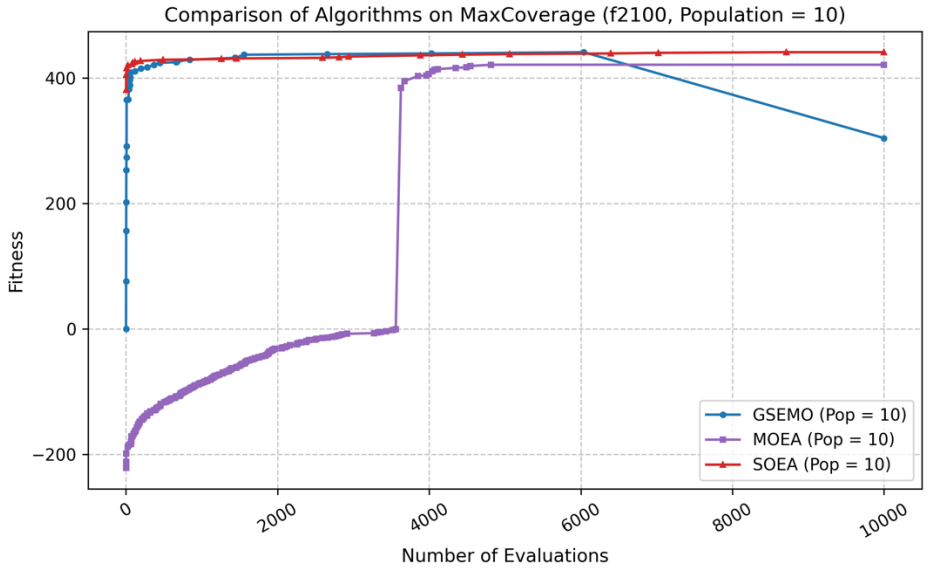

Figure 2: Comparison of GSEMO, MOEA, and SOEA on MaxCoverage (f2100, Population = 10).

- GSEMO performs most consistently and achieves the best convergence.
- MOEA shows unstable early behaviour but eventually improves.
- SOEA maintains stable, high performance with fewer oscillations.

Fig3: GSEMO on Different Problems (MaxCoverage vs MaxInfluence)

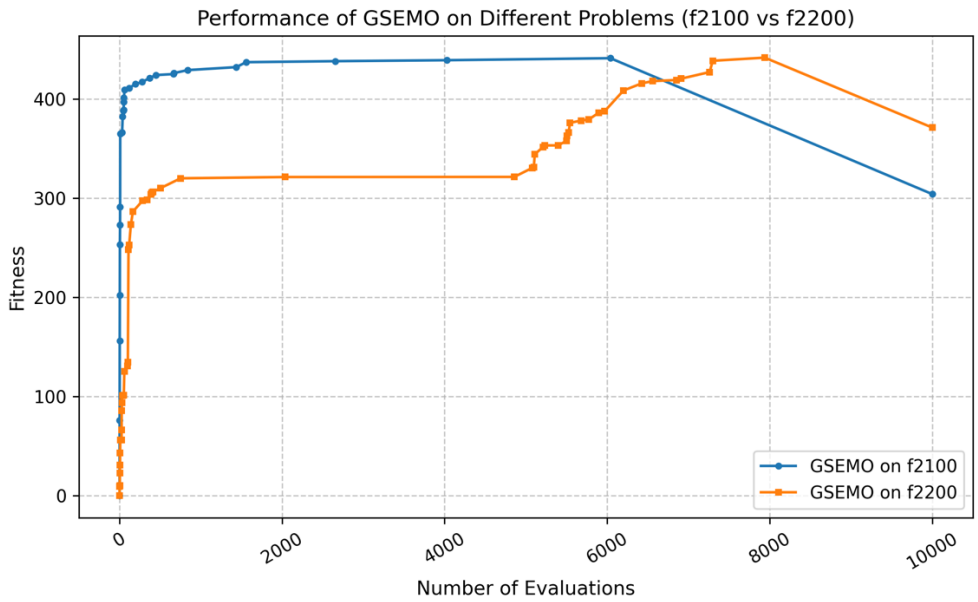

Figure 3: GSEMO performance on MaxCoverage (f2100) and MaxInfluence (f2200).

- MaxCoverage reaches higher fitness faster.
- MaxInfluence converges more slowly, suggesting it is a more complex instance.
- Both problems show that GSEMO effectively adapts to monotone submodular structures.

Analysis

The increase in population size enhances diversity and stability, but it also slows down convergence during the early stages of the search.
Overall, GSEMO demonstrates superior performance, particularly in terms of final fitness and convergence stability.
The MaxInfluence instance appears to be more challenging due to its nonlinear structure and complex fitness landscape.

This exercise demonstrates the design and performance of both single-objective and multi-objective evolutionary algorithms for monotone submodular optimisation.
The experiments show that:

- GSEMO achieves strong and stable results on both MaxCoverage and MaxInfluence.
- MOEA and SOEA can be competitive, but their performance is highly sensitive to parameter tuning.
- Population diversity and constraint management are the key factors in achieving robust and effective optimisation performance.

Conclusion

In this exercise, we designed and tested both single-objective and multi-objective evolutionary algorithms for monotone submodular optimisation.

The results show that increasing the population size improves diversity and stability but slows down convergence at the beginning.

GSEMO achieved the best overall performance, while MOEA and SOEA also worked well with proper settings.

Overall, this task helped us understand how population and diversity affect algorithm behaviour, and how to balance exploration and convergence in evolutionary optimisation.

Use of Generative AI

Generative AI (ChatGPT) was used to help design algorithm structures, explain theoretical aspects, and assist in writing plotting scripts.